

Recipe App: Public Access Setup

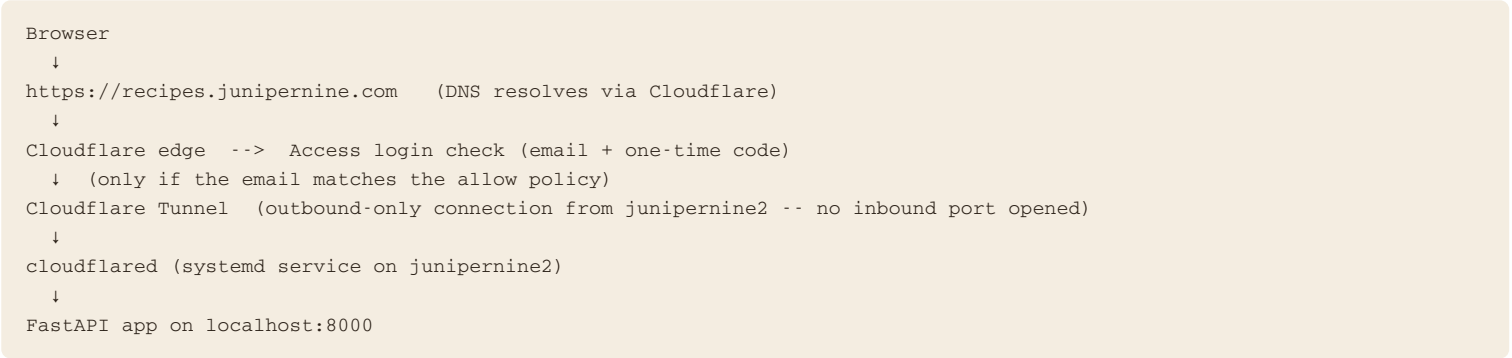
Cloudflare Tunnel + Cloudflare Access, for junipernine2 → recipes.junipernine.com

Documented September 2026 · project: junipernineaj/recipe-app

Overview

This document records how the recipe app (FastAPI + SQLite + HTMX, running on the home server `junipernine2`) was made reachable from the public internet at `https://recipes.junipernine.com`, without opening any port on the home router, and with a login wall in front of it so only approved people can reach the app at all.

The pieces involved, and how a request actually flows:



No changes were needed to the app's own code (`main.py`) to add authentication — Cloudflare Access sits entirely in front of the tunnel and blocks unauthenticated requests before they ever reach the app.

Prerequisites

- A domain already owned and able to have its nameservers changed (used here: `junipernine.com`, registered at GoDaddy)
- A free Cloudflare account
- Shell access to the server running the app (`junipernine2`), with `sudo`
- The app already running locally and reachable at a known port (here, `uvicorn` on port `8000`)

Step 1 — Point the domain at Cloudflare

On Cloudflare

1. Sign up at `dash.cloudflare.com` (Free plan is enough).
2. Click **Add a site**, enter the domain, select the Free plan.
3. Cloudflare scans existing DNS records (A, AAAA, MX, TXT, SRV, etc.) and imports them automatically. **Check MX and TXT records carefully if the domain is used for email** — a missed record here is the classic way email breaks during this step.
4. Cloudflare issues two nameservers, e.g. `deborah.ns.cloudflare.com` and `jasper.ns.cloudflare.com`.

On the registrar (GoDaddy)

1. My Products → the domain → DNS → Nameservers → Change Nameservers.
2. Switch to "Custom" and enter the two Cloudflare nameservers.
3. Save. Propagation can take minutes to ~24 hours (in practice, well under an hour).

If DNSSEC was ever enabled on the domain at the registrar, turn it off before or immediately after switching nameservers — leaving it on during a nameserver change can take the whole domain offline until resolved.

Cloudflare's dashboard flips the site from "Pending Nameserver Update" to "Active" once the change is detected globally.

Step 2 — Install cloudflared

On the server (Debian/Ubuntu-based), add Cloudflare's signed apt repository and install the client:

```
sudo mkdir -p --mode=0755 /usr/share/keyrings
curl -fsSL https://pkg.cloudflare.com/cloudflare-main.gpg | sudo tee /usr/share/keyrings/cloudflare-main.gpg >/dev/null
```

```
echo 'deb [signed-by=/usr/share/keyrings/cloudflare-main.gpg] https://pkg.cloudflare.com/cloudflared any main' | sudo tee
/etc/apt/sources.list.d/cloudflared.list
sudo apt-get update && sudo apt-get install cloudflared
```

Confirm it installed:

```
cloudflared --version
```

Step 3 — Authenticate cloudflared

```
cloudflared tunnel login
```

This prints a URL. On a headless server, copy that URL and open it manually on any machine that's logged into the Cloudflare account. Select the domain (e.g. `junipernine.com`) to authorize it. Approving in the browser saves a certificate back on the server, typically at `~/.cloudflared/cert.pem`.

Step 4 — Create the named tunnel

```
cloudflared tunnel create recipe-app
```

This generates a tunnel ID (a UUID) and writes a credentials file to `~/.cloudflared/<TUNNEL_ID>.json`. The tunnel ID is referenced in the config file in the next step.

Step 5 — Write the ingress config

Create `~/.cloudflared/config.yml`:

```
tunnel: <TUNNEL_ID>
credentials-file: /home/<user>/.cloudflared/<TUNNEL_ID>.json

ingress:
  - hostname: recipes.junipernine.com
    service: http://localhost:8000
  - service: http_status:404
```

The final catch-all rule (`http_status:404`) is required by `cloudflared` — any hostname not explicitly listed above it returns a 404 rather than being routed anywhere.

Step 6 — Route the DNS record

```
cloudflared tunnel route dns recipe-app recipes.junipernine.com
```

This automatically creates the CNAME record for `recipes.junipernine.com` pointing at the tunnel — no manual DNS editing needed.

Step 7 — Test manually before making it permanent

```
cloudflared tunnel run recipe-app
```

Leave this running in the foreground, then visit `https://recipes.junipernine.com` from a browser. It should load the app directly over HTTPS. Stop with `Ctrl+C` once confirmed.

Step 8 — Run it as a persistent service

```
sudo cloudflared --config /home/<user>/.cloudflared/config.yml service install
```

```
sudo systemctl start cloudflared
sudo systemctl enable cloudflared
sudo systemctl status cloudflared
```

Gotcha hit during setup: running plain `sudo cloudflared service install` (no `--config` flag) fails with *"Cannot determine default configuration path"*. This is because `sudo` runs the command as `root`, so `~/.cloudflared` resolves to `/root/.cloudflared` instead of the actual user's home directory where the config was created. Passing `--config` explicitly (as a global flag, before `service install`) fixes it. The install step then copies the config to `/etc/cloudflared/config.yml` for the `systemd` service to use.

`systemctl status` should show **active (running)**, and the app should still load at the public URL — now served by the background service rather than a terminal session, so it survives reboots.

Useful maintenance commands going forward:

Purpose	Command
Check service status	<code>sudo systemctl status cloudflared</code>
View live logs	<code>sudo journalctl -u cloudflared -f</code>
Restart after a config change	<code>sudo systemctl restart cloudflared</code>
Take the public URL offline temporarily	<code>sudo systemctl stop cloudflared</code>

Step 9 — Add the Cloudflare Access login wall

Everything above makes the app reachable publicly, but with **no authentication at all** — anyone with the URL could use it, including its delete functionality. This step closes that gap.

- In the Cloudflare dashboard, open **Zero Trust** (first visit prompts you to pick a plan — choose **Zero Trust Free**, intended for exactly this kind of home-lab use).
- Go to **Access → Applications → Add an application**.
- Choose **Self-hosted and private**, then the **Public DNS** destination type (not "Private destinations" — that option is for a different feature requiring the Cloudflare WARP client, and isn't needed here since the tunnel already serves a public hostname).
- Under **Destinations → Public hostnames**, set Subdomain to `recipes` and Domain to `junipernine.com`.
- Under **Access policies**, create a new policy: name it (e.g. "Email Access"), set Action to **Allow**, and under "Include" add rule(s) such as:
 - Emails** — specific individual addresses to allow
 - Emails ending in** — a domain wildcard (e.g. `junipernine.com`), so any address on that domain is allowed without listing each one
- Confirm **One-time PIN** is available as a login method (Cloudflare's built-in option — no external identity provider needed).
- Save the application.

The wizard's **"Connect a private web application"** option (which asks for an internal IP address and port) is a *different* feature — it sets up access via the WARP client to a private network destination. It was tried and skipped during this setup in favor of the simpler Public DNS + existing tunnel approach documented above.

Test in an incognito/private browser window: visiting `https://recipes.junipernine.com` should now show a Cloudflare Access login page first, asking for an email address, then a one-time code sent to it, before the app itself ever loads.

Adding or removing users later

Zero Trust → Access → Applications → the app → Access Policies → open the policy → add or remove an address under "Include: Emails". Changes apply immediately, live, with no need to restart the tunnel or anything on the server. Anyone whose address matches an existing domain wildcard rule doesn't need to be added individually at all.

Known non-issue encountered along the way

Early in setup, visiting the bare domain (`junipernine.com`, not the `recipes` subdomain) produced a Cloudflare **Error 1000: DNS points to prohibited IP**. This was an unrelated, pre-existing DNS record on the root domain pointing to a private/internal IP address while proxying was enabled — Cloudflare cannot proxy a public request to a private IP. It did not affect the `recipes.junipernine.com` subdomain or anything in this setup, and was left for separate cleanup.